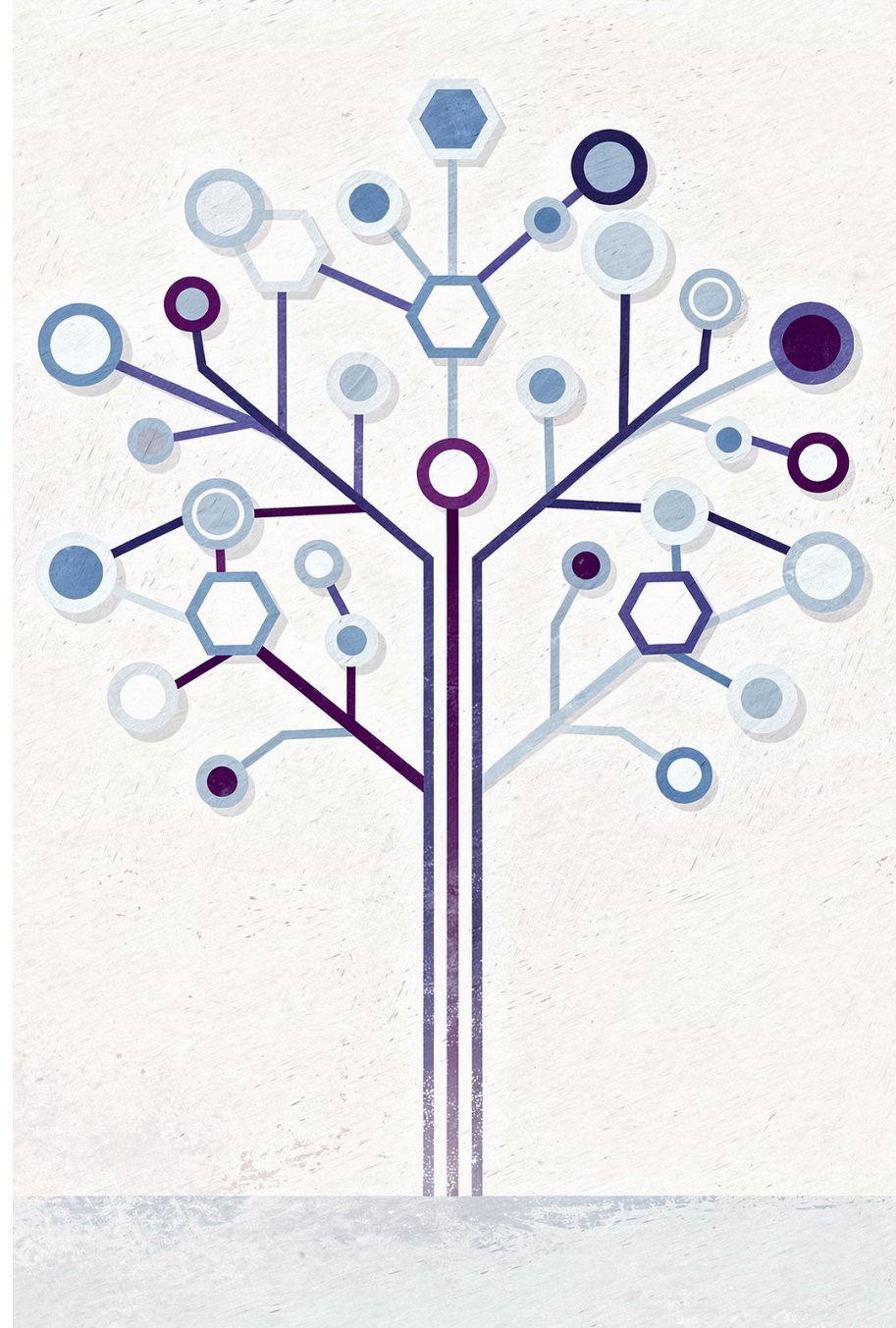


Patrón de Diseño Composite

También llamado: **Objeto compuesto** / **Object Tree**

Un patrón de diseño estructural que permite componer objetos en estructuras de árbol y trabajar con ellas como si fueran objetos individuales.



Recorrido de la Presentación

Explore con nosotros el Patrón Composite y su aplicación práctica.

01

Introducción

Definición y propósito del patrón.

03

Estructura y Conceptos

Diagrama de clases y roles de los participantes.

05

Consideraciones

Cuándo usarlo, pros y contras.

02

Caso de Uso

Un sistema de pedidos como ejemplo práctico.

04

Ejemplo Detallado

Implementación de un editor de formas gráficas.

06

Relaciones

Conexiones con otros patrones de diseño.



INTRODUCCIÓN

Propósito del Patrón Composite

Composite es un **patrón de diseño estructural** que te permite componer objetos en estructuras de árbol y trabajar con esas estructuras como si fueran objetos individuales.

Este patrón es fundamental cuando necesitas representar jerarquías parte-todo, permitiendo que los clientes traten de manera uniforme tanto a los objetos individuales como a las composiciones de objetos.

PROBLEMA

¿Cuándo tiene sentido usar Composite?

El uso del patrón Composite sólo tiene sentido cuando el **modelo central de tu aplicación puede representarse en forma de árbol**.

Por ejemplo, imagina que tienes dos tipos de objetos: **Productos** y **Cajas**. Una Caja puede contener varios Productos así como cierto número de Cajas más pequeñas. Estas Cajas pequeñas también pueden contener algunos Productos o incluso Cajas más pequeñas, y así sucesivamente.

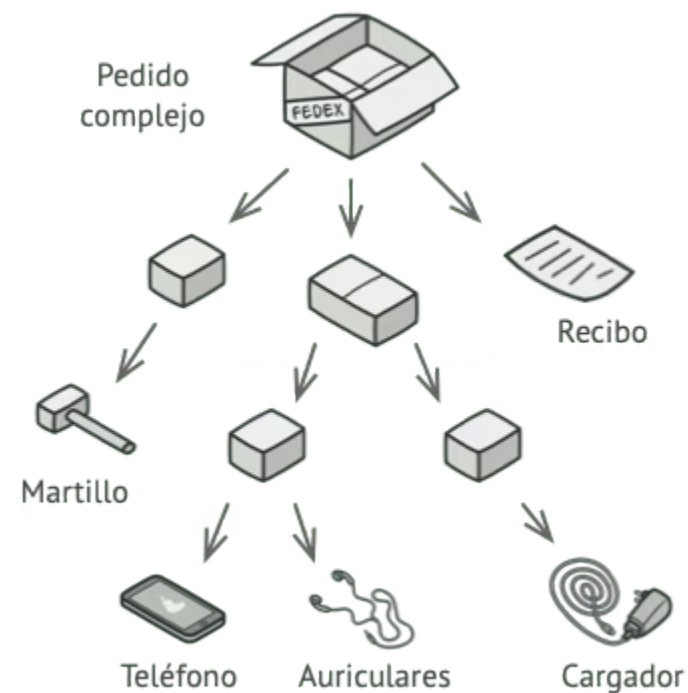
PROBLEMA

El Desafío del Sistema de Pedidos

Digamos que decides crear un **sistema de pedidos** que utiliza estas clases. Los pedidos pueden contener productos sencillos sin envolver, así como cajas llenas de productos... y otras cajas.

¿Cómo determinarás el precio total de ese pedido?

Un pedido puede incluir varios productos empaquetados en cajas, que a su vez están empaquetados en cajas más grandes y así sucesivamente. La estructura se asemeja a un **árbol boca abajo**.



PROBLEMA

¿Por qué la solución directa no funciona?

Puedes intentar la solución directa: desenvolver todas las cajas, repasar todos los productos y calcular el total. Esto sería viable en el mundo real; pero en un programa **no es tan fácil como ejecutar un bucle**.

Conocer clases de antemano

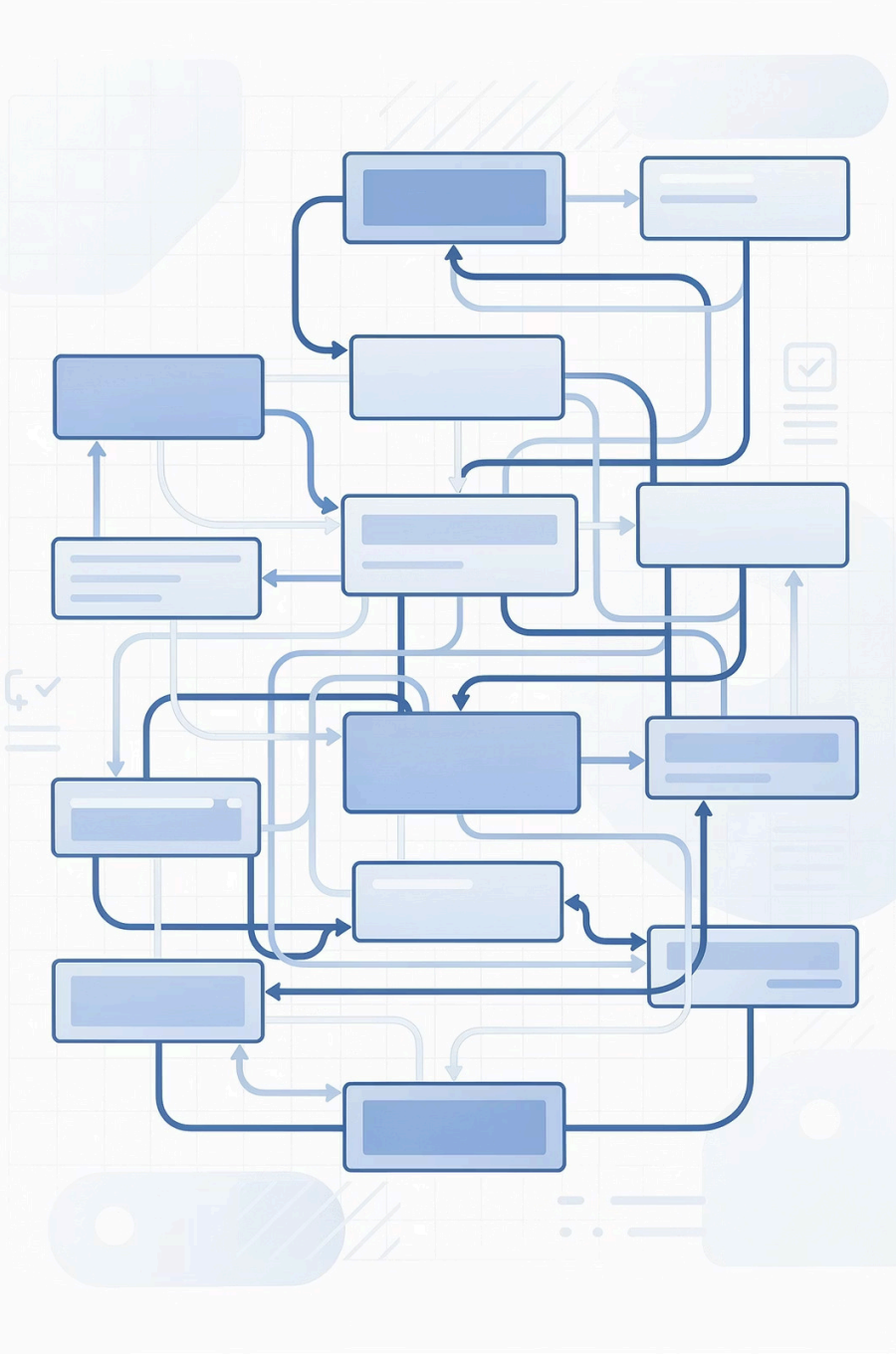
Tienes que conocer las clases de Productos y Cajas a iterar.

Nivel de anidación

Debes manejar el nivel de anidación de las cajas y otros detalles.

Complejidad excesiva

Todo esto provoca que la solución directa sea demasiado complicada, o incluso imposible.



SOLUCIÓN

La Propuesta del Patrón Composite

El patrón Composite sugiere que trabajes con Productos y Cajas a través de una **interfaz común** que declara un método para calcular el precio total.

Producto

Sencillamente devuelve el precio del producto.

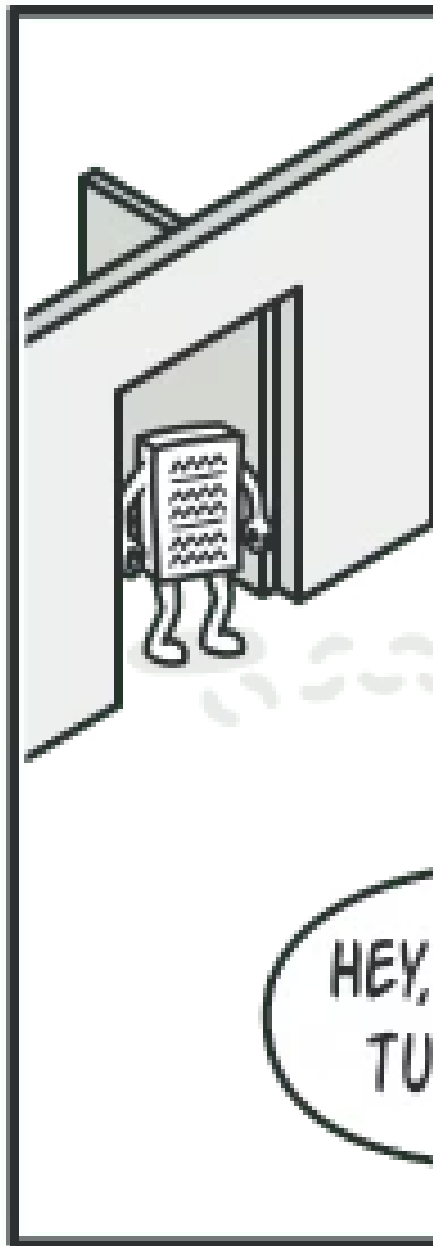
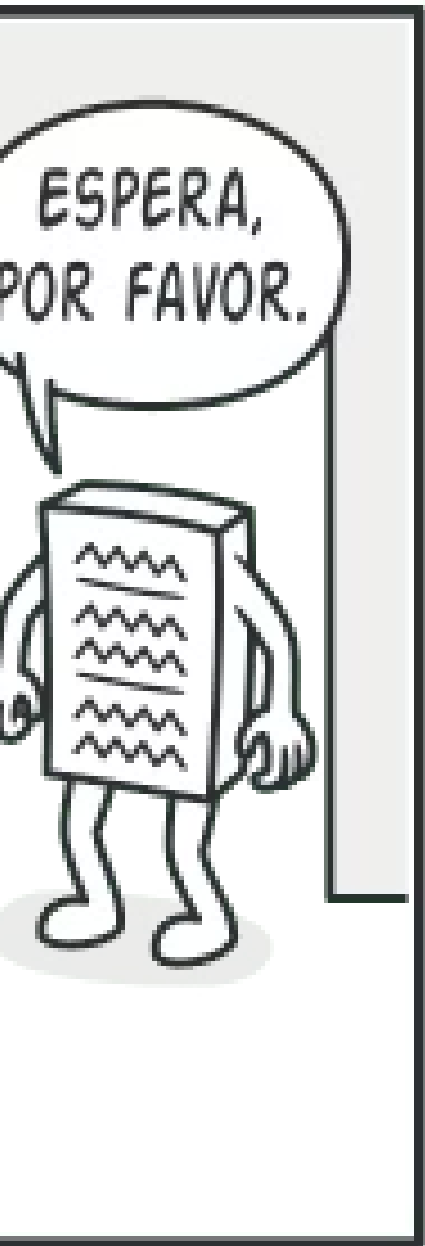
Caja

Recorre cada artículo que contiene, pregunta su precio y devuelve un total.

Recursión

Si un artículo es otra caja, esta también repasa su contenido, y así sucesivamente.

Una caja podría incluso añadir **costos adicionales** al precio final, como costos de empaquetado.



SOLUCIÓN

Comportamiento Recursivo en el Árbol

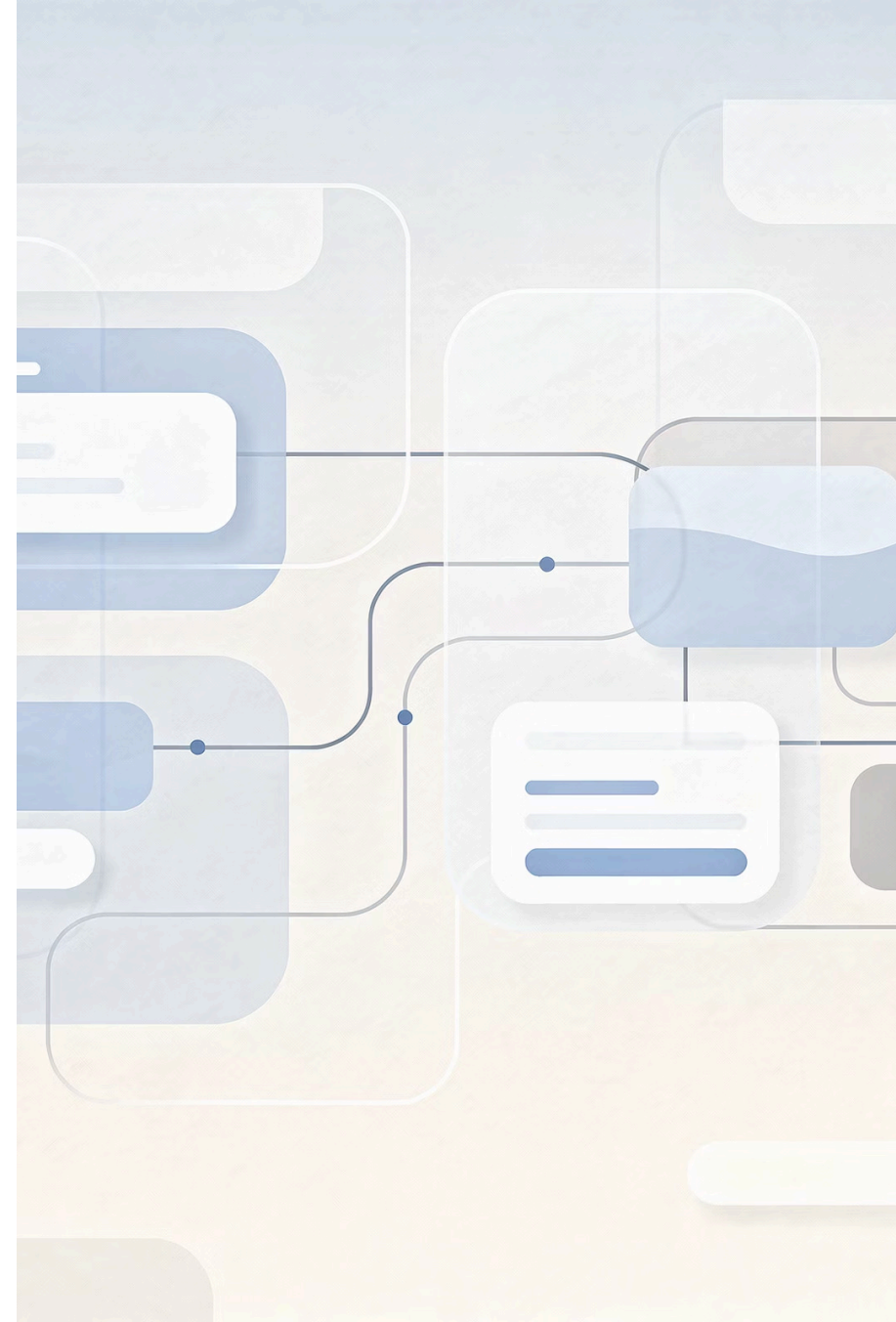
El patrón Composite te permite ejecutar un **comportamiento de forma recursiva** sobre todos los componentes de un árbol de objetos.

SOLUCIÓN

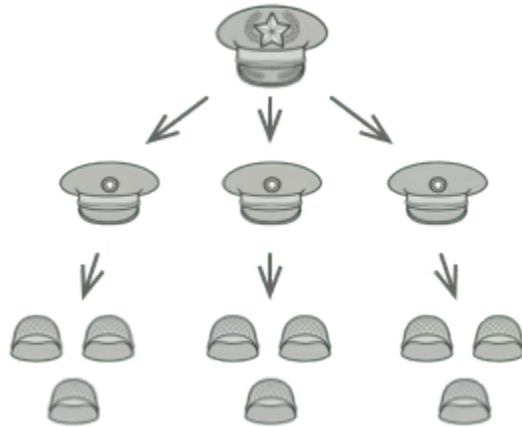
La Gran Ventaja

No tienes que preocuparte por las clases concretas de los objetos que componen el árbol.

No tienes que saber si un objeto es un producto simple o una sofisticada caja. Puedes **tratarlos a todos por igual** a través de la interfaz común. Cuando invocas un método, los propios objetos pasan la solicitud a lo largo del árbol.



Analogía en el Mundo Real



Estructura Militar

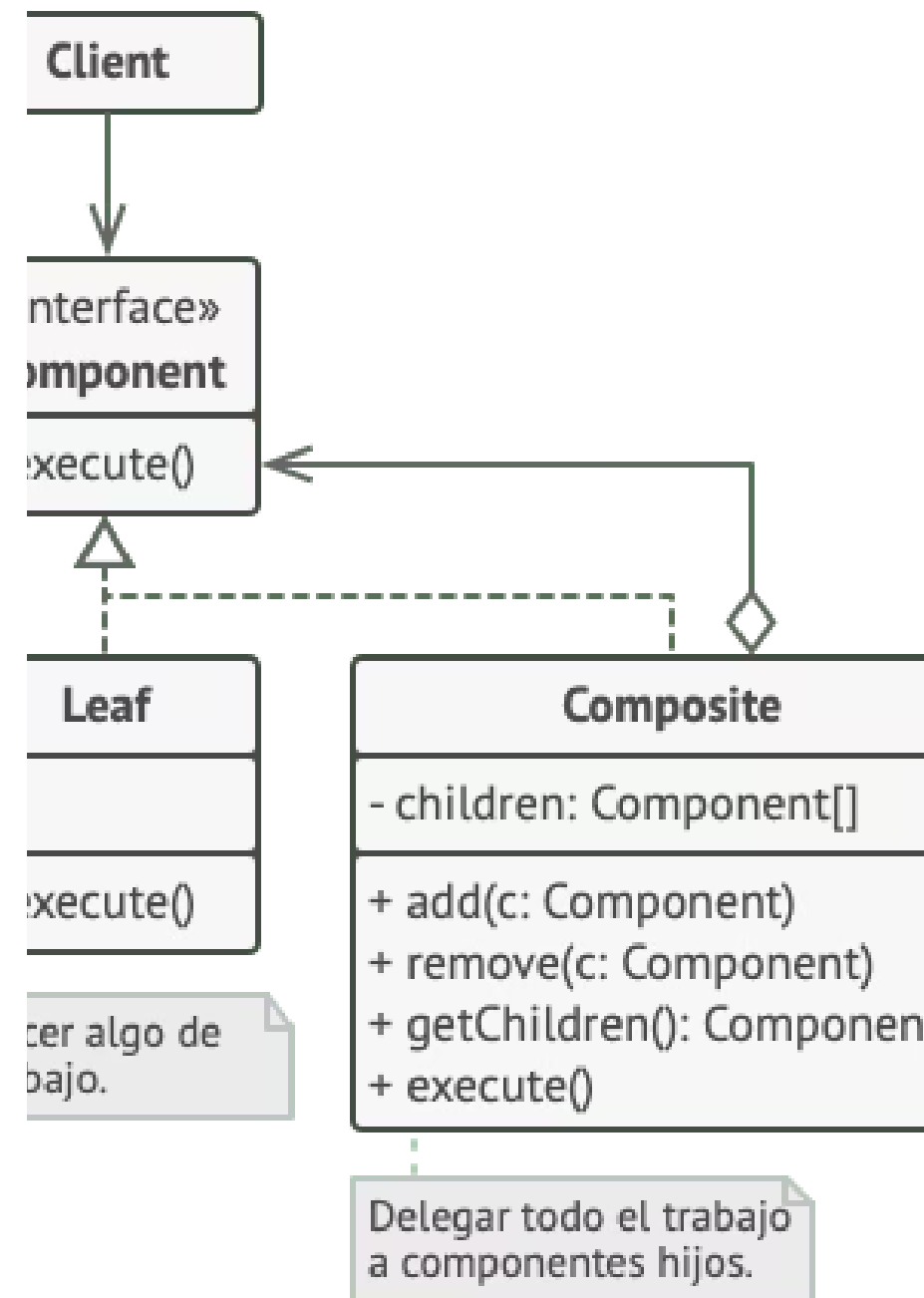
Los ejércitos de la mayoría de países se estructuran como **jerarquías**. Un ejército está formado por varias divisiones; una división es un grupo de brigadas y una brigada está formada por pelotones, que pueden dividirse en escuadrones.

Por último, un escuadrón es un pequeño grupo de **soldados reales**. Las órdenes se dan en la parte superior de la jerarquía y se pasan hacia abajo por cada nivel hasta que todos los soldados saben lo que hay que hacer.

ESTRUCTURA

Diagrama de Estructura

El diagrama muestra los cuatro participantes clave del patrón Composite: **Componente**, **Hoja**, **Contenedor** y **Cliente**.



Participantes del Patrón

1 Interfaz Componente

Describe operaciones comunes a elementos simples y complejos del árbol.

2 Hoja (Leaf)

Elemento básico de un árbol que no tiene subelementos. Normalmente, los componentes hoja acaban realizando la mayoría del trabajo real, ya que no tienen a nadie a quien delegarle el trabajo.

3 Contenedor (Composite)

Elemento que tiene subelementos: hojas u otros contenedores. No conoce las clases concretas de sus hijos. Funciona con todos los subelementos únicamente a través de la interfaz componente.

4 Cliente

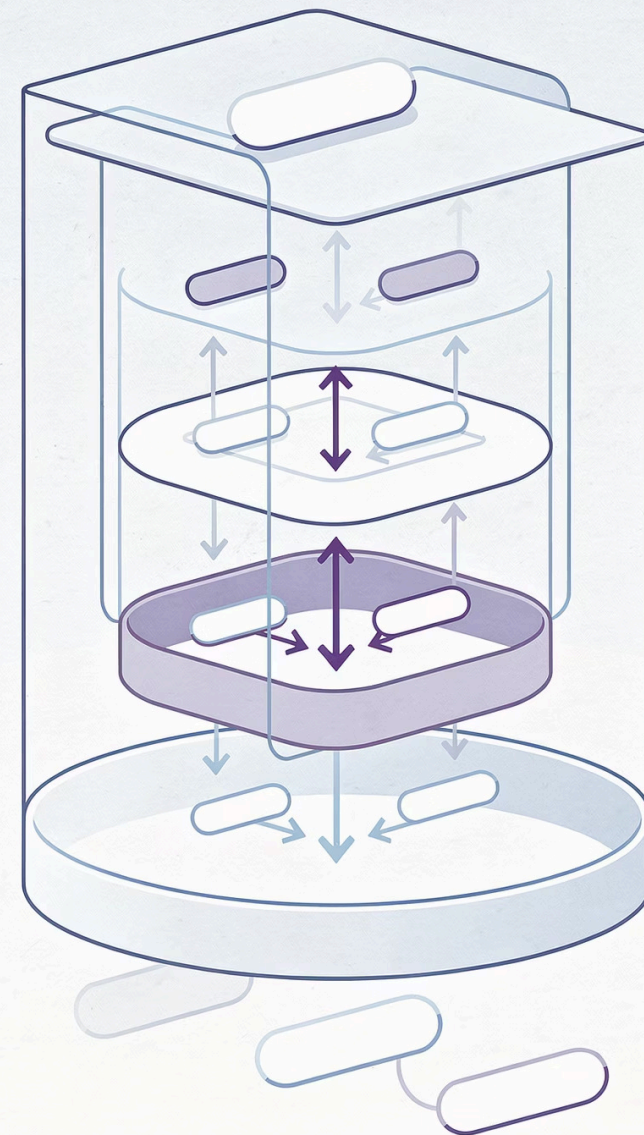
Funciona con todos los elementos a través de la interfaz componente. Puede funcionar de la misma manera tanto con elementos simples como complejos del árbol.

ESTRUCTURA

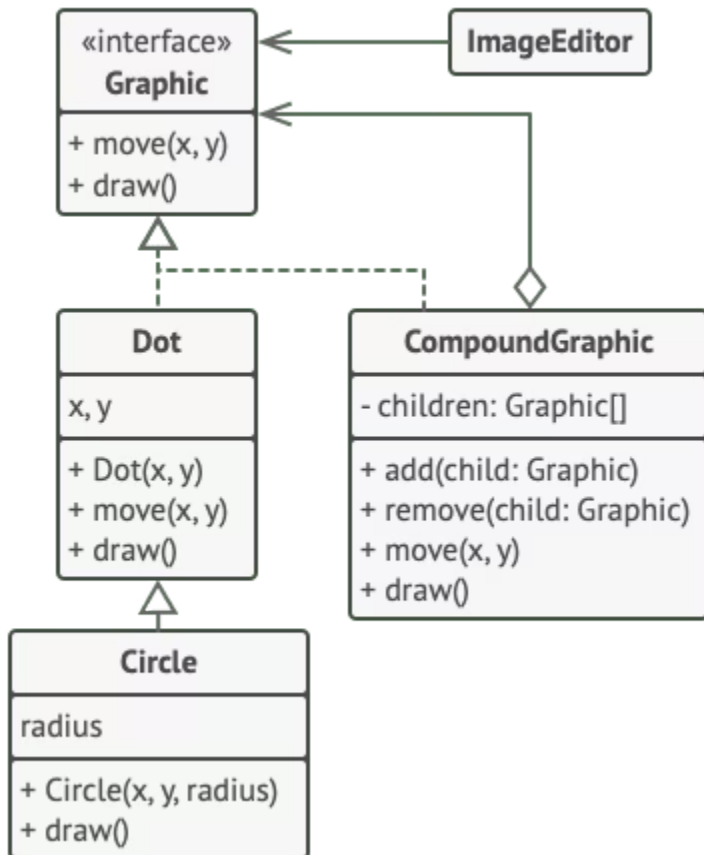
Comportamiento del Contenedor

Al recibir una solicitud, un contenedor **delega el trabajo a sus subelementos**, procesa los resultados intermedios y devuelve el resultado final al cliente.

El cliente puede funcionar de la misma manera tanto con elementos simples como complejos del árbol, gracias a la **interfaz componente compartida**.



Ejemplo: Editor de Formas Geométricas



En este ejemplo, el patrón Composite permite implementar el **apilamiento (stacking)** de formas geométricas en un editor gráfico.

La clase `GráficoCompuesto` es un contenedor que puede incluir cualquier cantidad de subformas, incluyendo otras formas compuestas. Una forma compuesta tiene los mismos métodos que una forma simple, pero pasa la solicitud de forma **recursiva** a todos sus hijos y "suma" el resultado.

PSEUDOCÓDIGO

Código Cliente del Editor

El código cliente trabaja con todas las formas a través de la **interfaz común** a todas las clases de forma. De este modo, el cliente no sabe si está trabajando con una forma simple o una compuesta. Puede trabajar con estructuras de objetos muy complejas **sin acoplarse** a las clases concretas que forman esa estructura.

Interfaz Graphic y Clase Dot

```
// La interfaz componente declara operaciones comunes para
```

```
// objetos simples y complejos de una composición.
```

```
interface Graphic is
```

```
    method move(x, y)
```

```
    method draw()
```

```
// La clase hoja representa objetos finales de una composición.
```

```
// Un objeto hoja no puede tener ningún subobjeto. Normalmente,
```

```
// son los objetos hoja los que hacen el trabajo real, mientras
```

```
// que los objetos compuestos se limitan a delegar a sus
```

```
// subcomponentes.
```

```
class Dot implements Graphic is
```

```
    field x, y
```

```
    constructor Dot(x, y) { ... }
```

```
    method move(x, y) is
```

```
        this.x += x, this.y += y
```

```
    method draw() is
```

```
        // Dibuja un punto en X e Y.
```


PSEUDOCÓDIGO

Clase Circle (Extensión de Dot)

```
// Todas las clases de componente pueden extender otros
// componentes.
class Circle extends Dot is
    field radius

    constructor Circle(x, y, radius) { ... }

    method draw() is
        // Dibuja un círculo en X y Y con radio R.
```

La clase `Circle` extiende `Dot`, demostrando que los componentes hoja pueden heredar de otros componentes simples, añadiendo funcionalidad específica como el radio.

Clase CompoundGraphic (Composite)

```
// La clase compuesta representa componentes complejos que  
// pueden tener hijos. Normalmente los objetos compuestos  
// delegan el trabajo real a sus hijos y después "recapitulan"  
// el resultado.
```

```
class CompoundGraphic implements Graphic is  
field children: array of Graphic
```

```
// Un objeto compuesto puede añadir o eliminar otros  
// componentes (tanto simples como complejos) a o desde su  
// lista hija.
```

```
method add(child: Graphic) is
```

```
// Añade un hijo a la matriz de hijos.
```

```
method remove(child: Graphic) is
```

```
// Elimina un hijo de la matriz de hijos.
```

```
method move(x, y) is
```

```
foreach (child in children) do
```

```
child.move(x, y)
```

Método draw() del CompoundGraphic

```
// Un compuesto ejecuta su lógica primaria de una forma
// particular. Recorre recursivamente todos sus hijos,
// recopilando y recapitulando sus resultados. Debido a que
// los hijos del compuesto pasan esas llamadas a sus propios
// hijos y así sucesivamente, se recorre todo el árbol de
// objetos como resultado.
method draw() is
    // 1. Para cada componente hijo:
    //   - Dibuja el componente.
    //   - Actualiza el rectángulo delimitador.
    // 2. Dibuja un rectángulo de línea punteada utilizando
    // las coordenadas de delimitación.
```

El método draw() recorre **recursivamente** todos los hijos, recopilando y recapitulando sus resultados. Se recorre todo el árbol de objetos como resultado.

Clase ImageEditor (Cliente)

```
// El código cliente trabaja con todos los componentes a través  
// de su interfaz base. De esta forma el código cliente puede  
// soportar componentes de hoja simples así como compuestos  
// complejos.
```

```
class ImageEditor is
```

```
  field all: CompoundGraphic
```

```
  method load() is
```

```
    all = new CompoundGraphic()
```

```
    all.add(new Dot(1, 2))
```

```
    all.add(new Circle(5, 3, 10))
```

```
  // ...
```

```
  // Combina componentes seleccionados para formar un
```

```
  // componente compuesto complejo.
```

```
  method groupSelected(components: array of Graphic) is
```

```
    group = new CompoundGraphic()
```

```
    foreach (component in components) do
```

```
      group.add(component)
```

```
    all.remove(component)
```

```
    all.add(group)
```

```
  // Se dibujarán todos los componentes.
```

```
  all.draw()
```

APLICABILIDAD

¿Cuándo Usar el Patrón Composite?



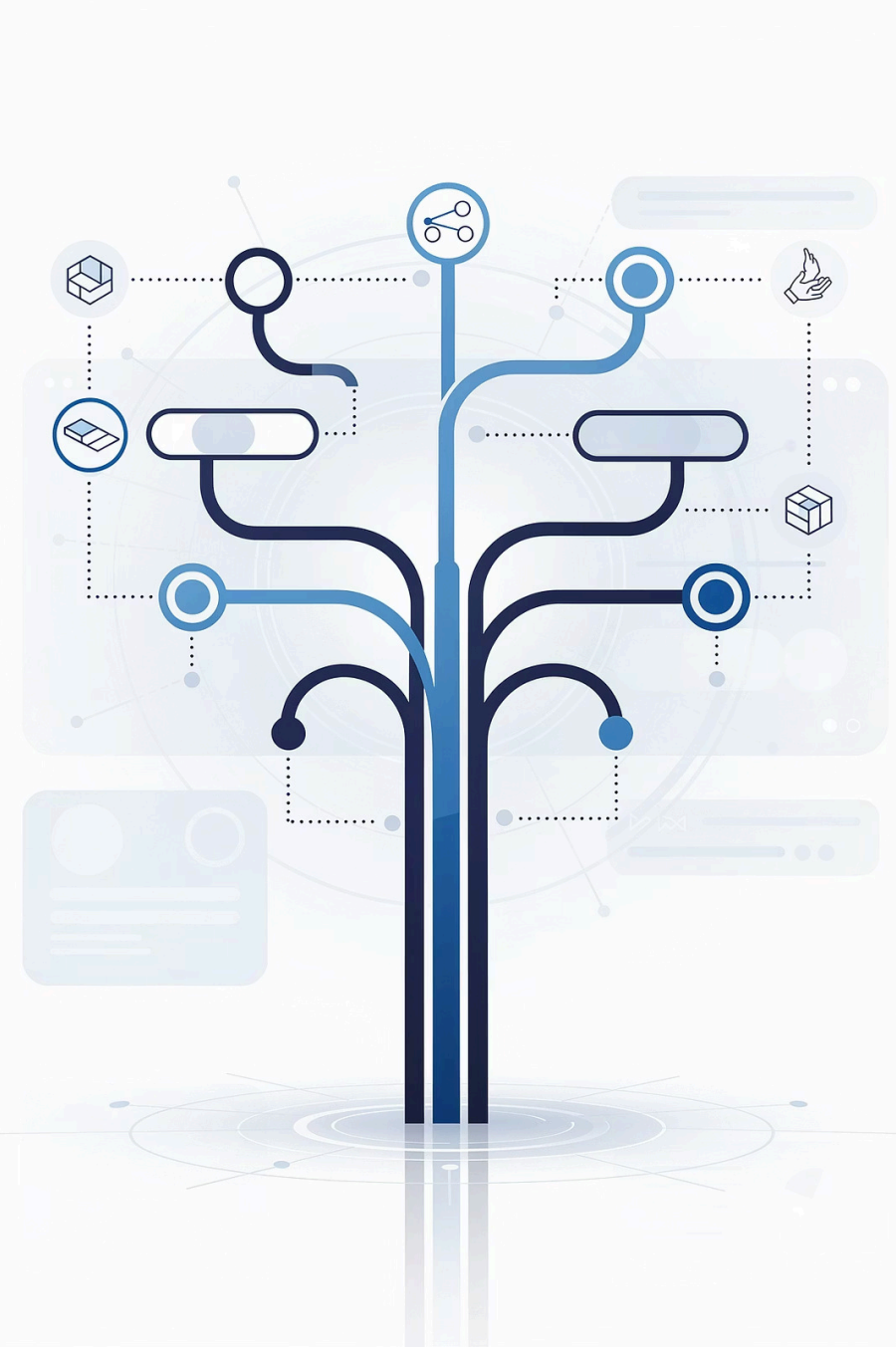
Estructura de Árbol

Utiliza Composite cuando tengas que implementar una estructura de objetos con forma de árbol. Te proporciona hojas simples y contenedores complejos que comparten una interfaz común.



Trato Uniforme

Utiliza el patrón cuando quieras que el código cliente trate elementos simples y complejos de la misma forma. El cliente no tiene que preocuparse por la clase concreta de los objetos.



Cómo Implementar el Patrón Composite

01

Identificar la estructura de árbol

Asegúrate de que el modelo central pueda representarse como un árbol. Divídelo en elementos simples y contenedores. Los contenedores deben poder contener tanto elementos simples como otros contenedores.

02

Declarar la interfaz componente

Declara la interfaz con una lista de métodos que tengan sentido para componentes simples y complejos.

03

Crear la clase hoja

Crea una clase hoja para representar elementos simples. Un programa puede tener varias clases hoja diferentes.

04

Crear la clase contenedora

Incluye un campo matriz para almacenar referencias a subelementos (hojas y contenedores). Al implementar los métodos, un contenedor debe delegar la mayor parte del trabajo a los subelementos.

05

Definir métodos de gestión de hijos

Define los métodos para añadir y eliminar elementos hijos dentro del contenedor.

Nota sobre la Interfaz Componente

- ❏ **Principio de segregación de la interfaz:** Las operaciones de añadir y eliminar hijos se pueden declarar en la interfaz componente. Esto violaría el principio porque los métodos de la clase hoja estarían vacíos. No obstante, el cliente podrá tratar a todos los elementos de la misma manera, incluso al componer el árbol.

Pros y Contras

✓ Ventajas

- Puedes trabajar con estructuras de árbol complejas con mayor comodidad: utiliza el **polimorfismo** y la **recursión** en tu favor.
- **Principio de abierto/cerrado.** Puedes introducir nuevos tipos de elemento en la aplicación sin descomponer el código existente, que ahora funciona con el árbol de objetos.

✗ Desventajas

- Puede resultar difícil proporcionar una **interfaz común** para clases cuya funcionalidad difiere demasiado.
- En algunos casos, tendrás que generalizar en exceso la interfaz componente, provocando que sea **más difícil de comprender**.

Relaciones con Otros Patrones



Builder

Puedes utilizar Builder al crear árboles Composite complejos, programando sus pasos de construcción para que funcionen de forma recursiva.



Chain of Responsibility

Se utiliza a menudo junto a Composite. Un componente hoja puede pasar la solicitud a lo largo de la cadena de todos los componentes padre hasta la raíz del árbol.



Iterator

Puedes utilizar Iteradores para recorrer árboles Composite.



Visitor

Puedes utilizar Visitor para ejecutar una operación sobre un árbol Composite entero.

Composite, Decorator, Flyweight y Prototype



Flyweight

Puedes implementar nodos de hoja compartidos del árbol Composite como Flyweights para **ahorrar memoria RAM**.



Decorator

Composite y Decorator tienen diagramas de estructura similares (composición recursiva). Un Decorator es como un Composite pero sólo tiene **un componente hijo**. Decorator añade responsabilidades adicionales, mientras que Composite "recapitula" resultados de sus hijos. Pueden colaborar: usa Decorator para extender el comportamiento de un objeto del árbol Composite.



Prototype

Los diseños que hacen un uso amplio de Composite y Decorator a menudo pueden beneficiarse del Prototype. Permite **clonar estructuras complejas** en lugar de reconstruirlas desde cero.